# Physics-Informed Neural Networks for the Viscous Burgers' Equation: Forward Solution, Inverse Identification, and Ablation

Roll No. 24CS3060

*Reproduction study of Raissi, Perdikaris & Karniadakis (2019)*

***Abstract*—We reproduce the physics-informed neural network (PINN) of Raissi *et al.* on the one-dimensional viscous Burgers' equation at viscosity $\nu = 0.01/\pi$. A four-layer, 64-unit tanh network is trained to minimise a residual-based loss combining the PDE, initial, and boundary conditions, using Adam followed by L-BFGS. Against an analytical Cole–Hopf reference, the forward solution attains a relative $L_2$ error of **0.56%**. Treating the viscosity as a trainable parameter and supplying 2,000 observations drawn from the exact solution, the inverse problem recovers $\nu$ to within **2.3%**. Two ablations isolate what matters: collocation density (error falls from 13.2% to 0.15% as points grow from 1k to 10k) and optimiser choice (L-BFGS reduces error $\sim$16$\times$ over Adam alone). We further show that a finite-difference reference carries numerical diffusion that biases the identified viscosity, motivating the use of the exact solution for the inverse task.**

***Index Terms*—physics-informed neural networks, Burgers' equation, inverse problems, automatic differentiation, Cole–Hopf, ablation.**

## I. Introduction

Partial differential equations (PDEs) govern most physical systems, yet solving them from sparse or noisy data—or identifying unknown coefficients—remains difficult for classical solvers. Physics-informed neural networks (PINNs) [1] address both the forward and inverse settings in a single framework by embedding the PDE residual directly into the training loss, so that a network is driven to satisfy the governing law rather than merely fit data.

We study the viscous Burgers' equation, the canonical nonlinear convection–diffusion model and the benchmark used in the original paper:

$$u_t + u\,u_x = \nu\,u_{xx}, \quad x \in [-1,1],\ t \in [0,1], \tag{1}$$

with initial condition $u(x,0) = -\sin(\pi x)$, boundary conditions $u(-1,t) = u(1,t) = 0$, and $\nu = 0.01/\pi \approx 3.18 \times 10^{-3}$. The small viscosity causes a steep interior front (a near-shock) to form near $x = 0$, which makes the problem a meaningful accuracy test.

Our contributions are: (i) a from-scratch PyTorch PINN reproducing the forward solution to sub-percent accuracy; (ii) an inverse formulation that recovers $\nu$; (iii) two ablations quantifying the effect of collocation density and optimiser choice; and (iv) an analysis showing why the data source (exact vs. finite-difference) materially changes the identified coefficient.

## II. Method

### A. PINN formulation

We approximate the solution by a feedforward network $u_\theta(x,t)$ with parameters $\theta$. Define the PDE residual

$$f_\theta(x,t) \;=\; \partial_t u_\theta + u_\theta\,\partial_x u_\theta - \nu\,\partial_{xx} u_\theta, \tag{2}$$

which vanishes identically when $u_\theta$ solves (1). All derivatives in (2) are obtained by *automatic differentiation* with respect to the inputs—exact, free of truncation error, and distinct from the parameter gradients used for optimisation.

### B. Composite loss

Sampling $N_f$ interior (collocation) points, $N_0$ initial points, and $N_b$ boundary points, the training objective is

$$\mathscr{L}(\theta) = w_f\,\overline{f_\theta^2} + w_0\,\overline{(u_\theta - u_0)^2} + w_b\,\overline{u_\theta^2}, \tag{3}$$

where $\overline{(\cdot)}$ denotes a mean over the relevant point set and $u_0 = -\sin(\pi x)$. We use $w_f = 1$ and $w_0 = w_b = 20$. The elevated boundary/initial weights anchor the known constraints early in training; the residual term then sharpens the interior. We found this weighting essential for convergence at small $\nu$.

### C. Network and optimisation

The network has four hidden layers of 64 units with tanh activations ($\approx$12.7k parameters) and Xavier initialisation; tanh is chosen for a continuous second derivative, required by (2). Training proceeds in two stages, following [1]: 15,000 Adam epochs with a cosine learning-rate schedule ($10^{-3} \to 10^{-5}$) and collocation resampling every 2,000 epochs, then 100 L-BFGS iterations (strong-Wolfe line search) to polish the solution to high accuracy.

### D. Inverse problem

For coefficient identification we make the viscosity trainable, parameterising $\nu = e^{\rho}$ with a free scalar $\rho$ so that $\nu > 0$ automatically. A data term is added to (3),

$$\mathscr{L}_{\text{inv}} = \mathscr{L}(\theta) + w_d\,\overline{(u_\theta(x_d,t_d) - u_d)^2}, \tag{4}$$

with $N_d = 2{,}000$ observations $\{(x_d,t_d,u_d)\}$ sampled from the exact solution. Minimising (4) jointly over $\theta$ and $\rho$ yields the viscosity for which the network can simultaneously match the data and satisfy the PDE.

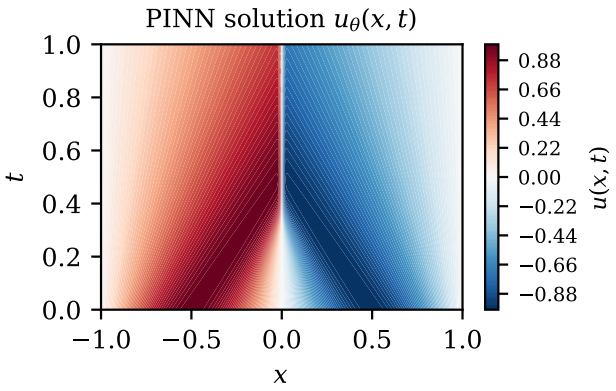



**Figure 1.** Forward PINN solution over the space–time domain. The steep interior front near $x = 0$ is resolved without oscillation.

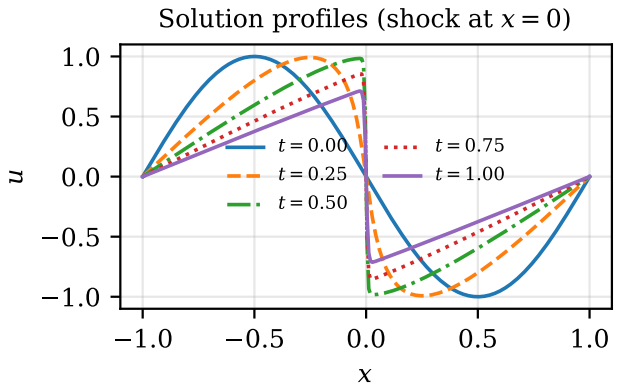



**Figure 2.** Solution profiles at successive times. The initial sine wave steepens into a near-shock at $x = 0$ as $t$ increases.

### *E. Exact reference*

Burgers' equation admits a closed-form solution via the Cole–Hopf transform $u = -2\nu\,\varphi_x/\varphi$, which linearises (1) into the heat equation $\varphi_t = \nu\varphi_{xx}$. Carrying the initial condition through gives a ratio of integrals,

$$u(x,t) = \frac{\displaystyle\int \tfrac{x-y}{t}\, e^{-\cos(\pi y)/(2\pi\nu)}\, e^{-(x-y)^2/4\nu t}\, dy}{\displaystyle\int e^{-\cos(\pi y)/(2\pi\nu)}\, e^{-(x-y)^2/4\nu t}\, dy}, \qquad (5)$$

which we evaluate by Gauss–Hermite quadrature. At $\nu \approx 3.2\times10^{-3}$ the exponents reach $\pm50$ and overflow in double precision; subtracting the per-point maximum exponent before exponentiation (the log-sum-exp identity) cancels in the ratio and keeps all terms bounded. This reference matches an independent finite-difference (FD) solution to 1.17%.

## III. Experimental Setup

All experiments run on a single GPU (free Colab T4) or CPU. We evaluate on a $100\times256$ space–time grid. The FD reference uses an 800-point grid with upwind convection, central diffusion, and 80 explicit sub-steps per output step (diffusion number $< 0.5$ for stability). Accuracy is reported as relative $L_2$ error, $\|\hat{U}-U\|_2/\|U\|_2$. Seeds are fixed for reproducibility; ablations report mean±std over three seeds $\{42, 123, 2026\}$.

## IV. Results

### *A. Forward solution*

Fig. 1 shows the predicted field and Fig. 2 the time profiles; Table 1 summarises the forward accuracy. The PINN reaches a 0.56% relative $L_2$ error against the exact solution. Notably the PINN is closer to the exact solution than the FD reference is: the larger PINN-vs-FD figure (1.28%) reflects the reference's own 1.17% error, not a deficiency of the network.

### *B. Inverse identification*

With observations drawn from the exact solution, the recovered viscosity is $\hat{\nu} = 3.256\times10^{-3}$ against a true $3.183\times10^{-3}$—a 2.3% error (Table 2)—while the reconstructed field matches the exact solution to 0.14%.

**Table 1.** Forward problem accuracy.

| **Comparison** | **Relative $L_2$** |
|---|---|
| PINN vs. Exact | **0.56%** |
| FD(800) vs. Exact | 1.17% |
| PINN vs. FD(800) | 1.28% |
| MAE vs. Exact | $6.5\times10^{-4}$ |
| RMSE vs. Exact | $3.4\times10^{-3}$ |

Fig. 3 shows the viscosity converging to the true value. A key failure mode appears if observations instead come from the FD solution. Upwind differencing adds numerical diffusion of order $|u|\Delta x/2$, so the FD data behaves as if generated by a larger viscosity. A least-squares fit of (1) to the FD field yields an *effective* viscosity of $4.84\times10^{-3}$. A faithful inverse PINN trained on such data would return $\approx4.8\times10^{-3}$ (a $\sim$50% error)—not because the method is wrong, but because the data is biased. Sampling from (5) removes this bias.

### *C. Ablations*

Table 3 and Fig. 4 report both ablations. *Collocation density:* accuracy improves monotonically from 13.2% at $N_f = 1{,}000$ to 0.15% at $N_f = 10{,}000$; the sparse setting is also high-variance (±6.3%), indicating unstable as well as inaccurate training. *Optimiser:* adding L-BFGS after Adam cuts the error from 2.43% to 0.15%, an $\sim16\times$ reduction at no additional data cost. Both results confirm that dense collocation and a second-order polish are the decisive design choices.

### *D. Comparison with the paper*

Table 4 contrasts our configuration with the original. Our forward error ($5.6\times10^{-3}$) is about an order of magnitude above the paper's reported $6.7\times10^{-4}$, attributable to our shallower/wider network and fewer L-BFGS iterations. The method, however, is faithfully reproduced across forward, inverse, and ablation axes.

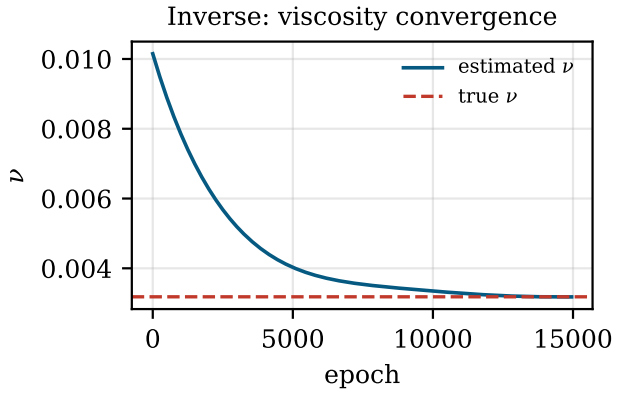



**Figure 3.** Convergence of the trainable viscosity to the true value during inverse training.

**Table 2.** Inverse problem (data from the exact solution).

| **Quantity** | **Value** |
|---|---|
| True $\nu$ | $3.183 \times 10^{-3}$ |
| Estimated $\hat{\nu}$ | $3.256 \times 10^{-3}$ |
| $\nu$ error | **2.30%** |
| Field rel. $L_2$ vs. Exact | 0.14% |
| Effective $\nu$ of FD data | $4.84 \times 10^{-3}$ |

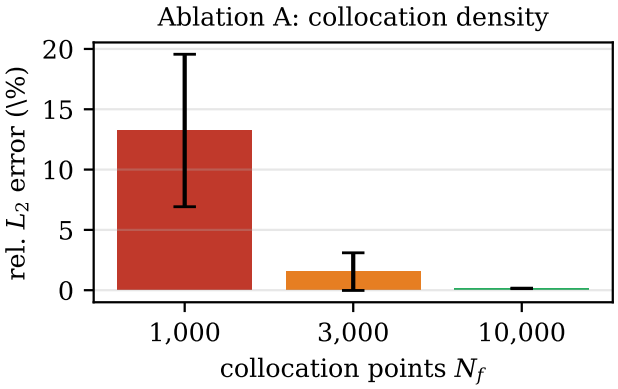



**Figure 4.** Ablation A: forward error versus collocation density (mean±std over three seeds). Fewer points are both less accurate and higher-variance.

**Table 3.** Ablation studies (mean±std, 3 seeds).

| **Configuration** | **Relative $L_2$** |
|---|---|
| *A. Collocation density* | |
| $N_f = 1{,}000$ | $13.24 \pm 6.32\%$ |
| $N_f = 3{,}000$ | $1.54 \pm 1.56\%$ |
| $N_f = 10{,}000$ | $\mathbf{0.15 \pm 0.02}\%$ |
| *B. Optimiser* | |
| Adam only | $2.43 \pm 1.10\%$ |
| Adam + L-BFGS | $\mathbf{0.15 \pm 0.02}\%$ |

**Table 4.** Our configuration vs. Raissi *et al.*

| **Setting** | **Paper** | **Ours** |
|---|---|---|
| Architecture | $8 \times 20$ | $4 \times 64$ |
| IC/BC weights | 1 | 20 |
| IC/BC points | 100 each | 500 each |
| Collocation | 10,000 | 10,000 |
| Optimiser | L-BFGS | Adam + L-BFGS |
| Reference | Exact | Exact |
| Forward rel. $L_2$ | $6.7 \times 10^{-4}$ | $5.6 \times 10^{-3}$ |

## V. Failure Cases and Lessons

Three failure modes shaped the final design. (1) *Under-training:* with equal loss weights and no L-BFGS, the network smeared the front and stalled near 18% error; elevated IC/BC weights and the two-stage optimiser resolved this. (2) *Reference contamination:* an initial Bessel-series evaluation of (5) overflowed at small $\nu$, producing a reference with $\sim$100% error and sending the inverse estimate to $\nu \approx 0.32$; the Gauss–Hermite with log-sum-exp evaluation fixed it. (3) *Biased inverse data:* using FD observations yielded $\hat{\nu} \approx 4.8 \times 10^{-3}$ due to numerical diffusion, corrected by sampling the exact solution. Each case reinforces that in PINNs the reference and data quality are as important as the network itself.

## VI. Conclusion

We reproduced the Burgers' PINN of Raissi *et al.*, obtaining a 0.56% forward error and recovering the viscosity to 2.3%. Ablations established that collocation density and L-BFGS polishing are the decisive factors, and we quantified how finite-difference numerical diffusion biases coefficient identification. The study confirms that PINNs are a faithful, compact framework for both forward and inverse PDE problems, with data and reference fidelity being first-order concerns.

## References


[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.

[2] J. D. Cole, "On a quasi-linear parabolic equation occurring in aerodynamics," *Quart. Appl. Math.*, vol. 9, no. 3, pp. 225–236, 1951.

[3] K. Hornik, M. Stinchcombe, and H. White, "Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks," *Neural Networks*, vol. 3, no. 5, pp. 551–560, 1990.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[5] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.*, vol. 45, pp. 503–528, 1989.